Australian National University

Lab Four

Finite State Machines

Signal interfaces

(v2026.2: updated class-use manual)

Overview

Finite state machines (FSMs) control the sequential execution of functions based on a system's state and inputs.

In this lab, you will

- Design an FSM to mimic the operation of a Microwave oven's control system.
- Design and implement a FSM-controlled system to receive and decode an incoming signal containing a serial data stream.

© ANU 2026

Australian National University

Table of Contents

Learning Outcomes 2
Activity 1: Microwave Control System [1 hour, 7 marks] 3
Procedure 4
Step 1: Create a Vivado project 4
Step 2: Create an FSM module inside the project 4
Step 3: Declare state variables and assign values to each state 4
Step 4: Describe the Current State Register/ Transition block 4
Step 5: Next State Logic 4
Step 6: Fill in the Next State Logic for COOK State 5
Step 7: Output Logic 5
Step 8: Module interconnect in TOP module 6
Step 9: Constraints file for your TOP module 6
Step 10: Synthesise – Implement – Generate Bitstream – Program the board 6
Activity 2: Interpreting an incoming serial signal 7
Part 1: Human as the transmitter [1.5hr, 8 marks] 7
Step 1: assemble the system datapath from the available building blocks 9
Step 2: describe the behaviour of the system as a set of states with transitions, and design a FSM controller 9
Step 3: add the FSM, review and complete your block diagram 10
Step 4: Create a Vivado project and assemble the design 10
Step 5: Implement and test 10
Part 2: A digital signal generator as a source [30 min, 5 marks] 11
Step 1: plan the necessary changes to the design 11
Step 2: copy and modify the Part 1 design 12
Step 3: simulate your design 12
Step 4: Implement and test your design 12
Appendix: Trying this lab on a Basys3 board 13

Learning Outcomes

By completing this lab, you will learn to:

1. Instantiate modules in hierarchical designs.
2. Use a combination of resets, enables and finite state machines to control sequential logic processes.

© ANU 2026

Activity 1: Microwave Control System [1 hour, 7 marks]

A Finite State Machine (FSM) controls the flow and execution sequence of a system. An FSM divides the control sequence of an application into discrete, interconnected states. These states determine which stage of operation the system is in, based on which the same inputs may have different effects/outputs. An FSM is described using a state diagram and a state transition table. FSMs can be of two main types, Moore and Mealy Machines.

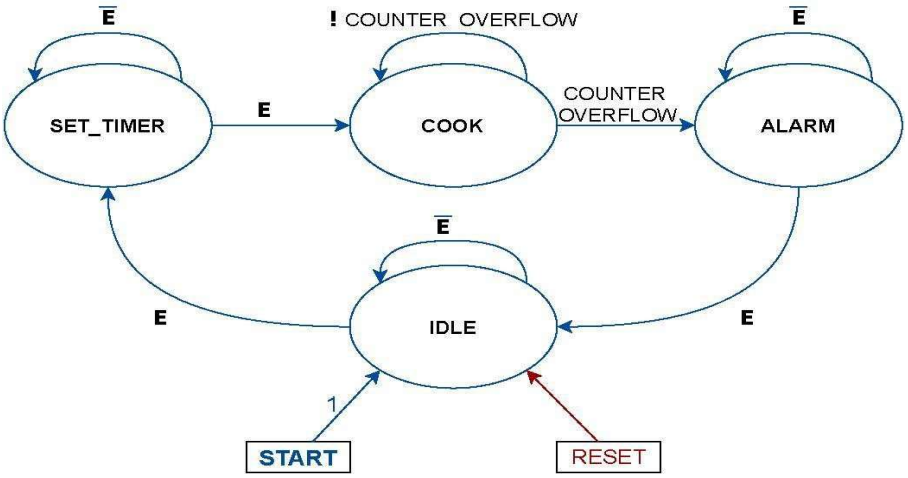

Figure 1: State Diagram of the microwave oven controller.

In this Activity, the FSM operates as a microwave oven controller. Figure 1 shows the state diagram.

We shall use the LEDs as the outputs and the switches and push buttons as inputs. The system tries to mimic the working of a microwave oven using the peripherals available on the development board. Thus, the system may be summed up as follows:

- **IDLE**: The system waits for user input. Press enter (**E** - a pushbutton) to start.
- **SET_TIMER**: Use the switches to set the timer by turning on the switch whose index corresponds to the number of seconds. E.g., Switch on SW[3] to set cooking time as 3 seconds. If multiple switches are set, the one corresponding to the longest time dominates.
- **COOK**: Wait until timer has elapsed.
- **ALARM**: Flash LEDs to indicate cooking process has completed. Press enter to restart (go back to IDLE).

© ANU 2026

Procedure

Step 1: Create a Vivado project

1. Create a new Project 'MicrowaveFSM' in Vivado.
2. Create a top-level module 'MicrowaveFSM_top.v'. This module will be used to connect all the different parts of the project. This module allows pre-processing of input signals (e.g., debounce, SPOT) before passing them to the FSM. The inputs are clk, reset, enter (**E** in the state diagram) and the switch array SW. The output is the LED array.

```
module MicrowaveFSM_top(
      input wire clk, enter, reset,
      input wire [3:0] SW,
      output wire [3:0] LED
);
```

We will fill in this module later.

Step 2: Create an FSM module inside the project

Create another source file called 'FSM.v'.

```
module FSM(
      input wire clk, enter, reset,
      input wire [3:0] SW,
      output reg [3:0] LED
);
```

The output LED array is **reg** so that it can be driven inside a procedural block.

Step 3: Declare state variables and assign values to each state

According to Figure 1, there are four states. Therefore, we shall use 2 bits to represent each state. Assign bit values to each state and declare them as parameters inside 'FMS.v' module.

```
parameter IDLE=2'd0,COOK=2'd1,SET_TIMER=2'd2,ALARM=2'd3;
```

Step 4: Describe the Current State Register/ Transition block

An FSM consists of 3 main parts. The Next State Logic, the Output Logic and the Current State Register/Transition block.

1. The Current State Register/Transition block stores the current state of the system and loads the appropriate next state at every clock cycle. Since 2 bits are used to represent each state, declare `state` and `nextstate` as 2-bit **reg**.

```
reg [1:0] state, nextstate;
```

2. Write a procedural block synchronised with `clk`. If `reset`, the `state` returns to IDLE; else, `state` takes the value of `nextstate`.

Step 5: Next State Logic

The next state logic is crucial in controlling the sequence of events and states. Next state logic

 © ANU 2026

depends on the inputs to the system and the current state.

Describe the next state logic using a procedural block. **Remember:** The next state logic **does not** need to be synchronised with `clk`. The next state logic is best described using combinatorial logic through a case statement.

```
always@(*)begin
      case(state)
          IDLE: begin
             if(enter) nextstate= SET_TIMER;
             else nextstate=IDLE;
           end
          SET_TIMER: begin
              ...
              ...
     endcase
end
```

Based on Figure 1, fill in the structure **apart from the next state logic for COOK state** (that's in the next step).

Step 6: Fill in the Next State Logic for COOK State

The system waits during the COOK state until a timer runs out before proceeding to the next ALARM state. We shall use a heartbeat generator to count the number of seconds.

1. Add 'clockDividerHB.v'. Instantiate this module with the appropriate parameter value to create a 1 Hz beat. Consider how to connect its `reset` and `enable` inputs so that the beat generation appropriately starts upon entry into the COOK state.

2. Declare a **reg** [3:0] ledcount. Write a procedural block synchronised with clk: during SET_TIMER state, the reg `ledcount` stores the input timer value (`SW`). During COOK state, right shift `ledcount` by 1 at every beat signal:

```
if(beat) ledcount <= ledcount >> 1;
```

3. Once the `ledcount` becomes empty, the cooking time will have elapsed and we can move to the ALARM state. In next state logic for COOK, you need the following:

```
if(&(~LED)) nextstate=ALARM;
```

*Note that `LED` is connected to `ledcount` in the output logic for COOK state.

Step 7: Output Logic

The output logic dictates the behaviour of the system that is controlled by the FSM.

1. In our system, we can set all LEDs low to indicate IDLE state. In SET_TIMER state, we connect `LED` to `SW` input directly to show the input timer value.

2. In COOK state, `LED` is connected to `ledcount`. We shift the LEDs right at every beat signal (see previous step) to indicate the amount of time left.

3. In ALARM state, `LED` is connected to `ledtoggle`. We toggle (flash) all four LEDs to indicate that the whole cycle is completed. LED toggling can be achieved by a T flip flop.

© ANU 2026

Please write a separate procedural block and reuse the 1Hz beat signal to toggle the LEDs.

```
always @(*) begin
      case(state)
            IDLE: LED = 4'b0000;
            SET_TIMER: LED = SW;
            COOK: LED = ledcount;
            ALARM: LED = ledtoggle;
            default: LED = 4'b1111;
      endcase
end
```

Step 8: Module interconnect in TOP module

The top module is very useful in assembling all the different blocks/components of the system and connecting them appropriately. The top module can also pre-process input signals to reduce noise, sample inputs at the right sampling frequency, buffer values for bulk processing, and sync various asynchronous inputs.

1. In the top module in 'MicrowaveFSM_top.v', debounce the *enter* button.
2. Use Single Pulse on Transition (SPOT) on the debounced *enter* button signal.

Then, instantiate your FSM module in the top module and pass the SPOT-processed *enter* signal and other raw inputs to the FSM.

Step 9: Constraints file for your TOP module

Link to Zybo Z7 constraints: digilent-xdc/Zybo-Z7-Master.xdc at master · Digilent/digilent-xdc · GitHub . You need constraints for the (system) clock, all LEDs, all switches, one push button for *enter*, and another push button for *reset*.

Step 10: Synthesise – Implement – Generate Bitstream – Program the board

Please demonstrate to the tutor the whole working cycle of the microwave. In your notebook, please sketch the block diagram of the design.

© ANU 2026

Activity 2: Interpreting an incoming serial signal

In this activity, you will develop a basic receiver system, which monitors an active signal, captures it, and interprets it. The behaviour we are implementing is typical of a serial communication receiver. Variations of it are commonly used when interfacing with external hardware.

When attempting communication between two distinct systems, a key challenge is that sender and receiver do not belong to the same *clock domain*[1]. Even when they run on the same nominal clock frequency (e.g., two boards, both with an advertised 100MHz clock), their clocks cannot be assumed to be perfectly identical, nor can we expect edge alignment. The result of this mismatch is that a receiver basing itself on its own clock risks reading the received information at the wrong time (e.g., violating timing constraints for registers), which could lead to data corruption.

In so-called *synchronous* serial communication, a communication clock signal, called *serial clock*, is shared between the two communicating systems. This signal allows the sender and the receiver to supply/read data according to an agreed trigger, which supports reliable communication.

The below figure provides a high-level overview of the process.

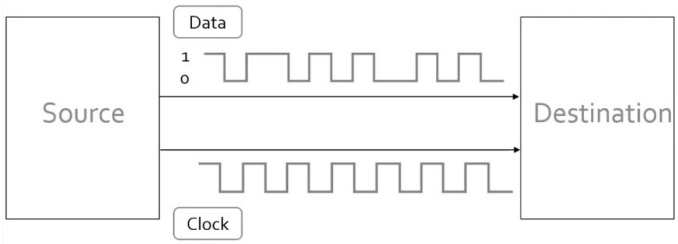

Figure 2: Synchronous serial communication uses a single data line and a second, shared "serial clock" line, which accurately times the capture of data by the receiver.

Part 1: Human as the transmitter [1.5hr, 8 marks]

We will start with an implementation where a human is acting as the transmitting entity. This allows us to develop a slow-paced system which is easier to conceptualise and debug. Since humans do not have a "signal clock" we can send to the receiver (the board), we will instead have the receiver "send" us a timing reference in the form of a slow heartbeat signal displayed via flashing LEDs. We already know about heartbeats from previous lab activities.

This activity requires you to design and implement the receiver system, governed by a state machine, which performs actions at discrete time steps, triggered by a reference 1 Hz heartbeat pulse.

The system behaviour is defined as follows:

- When the "capture" pushbutton is pressed, the system enters a capture phase.
- During capture, the system generates eight consecutive heartbeat pulses. Each heartbeat pulse represents the boundary of a time step. NOTE: The heartbeat is not a clock. In keeping with

[1] A *clock domain* is a system (or sometimes a sub-system) whose behaviour is associated with a common clock generation process – usually, a hardware oscillator.
Important clarification: a system using multiple clock frequencies that are *synchronously* derived from a common base clock constitutes one single clock domain. That's because all the switching edges remain aligned.

© ANU 2026

good synchronous FPGA design, all sequential logic must be clocked by the system clock. Use the heartbeat as an enabling pulse.

- **At each heartbeat** pulse:
    - If the "signal" pushbutton was pressed (by the transmitting human) *at any time* during the step, a logic 1 is captured, otherwise, 0 is captured. You will need to develop a way to capture and hold the button press event until the end of each step regardless of button release.
    - The sampled bit is shifted into an 8-bit shift register. The shift must occur once per heartbeat pulse, at the same moment as the logic capture for the relevant timestep.
    - A step counter is incremented.
- **After 8 steps**:
    - The captured 8-bit value is copied from the SIPO shift register to a holding register.
    - The system exits the capture phase.
    - The stored value is interpreted as an ASCII character and displayed as a single digit on the seven-segment display. Note: not all ASCII characters are displayable on a simple seven-segment display, so many 8-bit codes will return an "invalid" symbol – more on this later.
    - The displayed value remains unchanged until the next capture sequence has **completed**, at which point a new value is written into the holding register.

Other notable behaviours:

- LED outputs:
    - During the capture stage, two LEDs blink once alternately at each heartbeat pulse (railway crossing-style).
    - When the system is not capturing, both LEDs remain ON.
- Reset
    - A reset pushbutton is also available, to return the system to the pre-capture state. The behaviour of stored/displayed data after reset is not prescribed.

There are a few possible ways to approach this design. In order to keep the lab activity streamlined, you have been supplied with code for the following building blocks (Verilog modules), which you may use for your purposes:

Module	Inputs/outputs	Purpose / comments
ClockDividerHB	I: reset, enable, clock O: divided clock, heartbeat	Provides a timebase for the 1Hz behaviours of the system. Same code as previous labs.
Debouncer	I: reset, enable, clock, signal_in O: deb_out	Cleans up chatter from noisy buttons Same code as previous lab, but faster debounce.
SPOT	I: spot_in, clock O: spot_out	Rising edge detection module. Same code as previous labs.
Synchroniser	I: sync_in, clock O: sync_out	Aligns an incoming signal with system clock. Same code as previous labs.

 © ANU 2026

SIPO	I: serial_in, clock, push O: parallel_out	Collects input bits from a single signal line. These are shifted and stored when "push" is asserted. Register content can be read continuously from a parallel bus.
LED_driver	I: clock, blink_pulse, blink_active O: [1:0] LEDs	Drives two LEDs to alternate (railway crossing-style) in sync with a blink_pulse signal, or remain steady depending on selector signal blink_active. New code
ASCII2ssd	I: [7:0] ascii_code O: [6:0] ssd_sequence	Translates an incoming 8-bit sequence representing an ASCII character into a seven-segment display pattern (if the character can be rendered).
Holding_register	I: [0:SIZE-1] data_in, reset, enable O: [0:SIZE-1] reg_data	A basic parallel bank of flip-flops for data storage. It is parametrised so that the bitwidth can be set by adjusting parameter SIZE. Can be used for "signal" or display holding purposes in this design.
FSM	I: clock, reset, other inputs of your choice O: outputs of your choice	This module is to act as the control "brain" of the system, managing sequencing and operations of the other modules. You have been given a template but it is for you to design.

Please follow the below steps to complete the task:

Step 1: assemble the system datapath from the available building blocks

In your notebook, sketch a RTL diagram which interconnects the provided blocks to form a coherent *data path* from inputs to outputs. You can include additional blocks of your choice if you see a reason for doing so, though this should not be strictly necessary. If you are stuck, you can pause this task and work on Step 2 (FSM design) to see if this helps you structure your ideas, then return to Step 1.

Your chain of blocks should include signal conditioning (Debouncer? Synchroniser? SPOT?), the SIPO register for serial capture, the holding register for persistence, and the SSD and LED driving logic.

Some hints:

- Start with the 1Hz heartbeat as your timekeeping reference.
- Ensure that all edge-sensitive operations are clocked by the system clock; use heartbeat only as an enable/pulse signal.
- Make sure you have clearly resolved any data holding challenges (at the input button and for the SIPO output)
- Throughout this task, pay attention to the inputs and outputs associated with each of the given modules. Be sure to identify clocked and non-clocked components. Mark bus and single-bit lines to help you interpret your RTL diagram visually.

Step 2: describe the behaviour of the system as a set of states with transitions, and design a FSM controller

In your notebook, list and describe the different states of the proposed system, and sketch a state diagram (bubbles + labelled arrows).

A good way to ensure that all states are covered is to start from a known base state (e.g. "S0: idle") and identify which inputs, or combinations of inputs, would cause the system to transition to a different state. Repeat this process until you have covered all meaningful combinations of change triggers and operating conditions.

 © ANU 2026

A Finite State Machine controller module will be responsible for governing the overall behaviour of the system according to the states you have identified. The FSM controller module is connected to the transition-generating inputs, while its output logic drives signals which control the behaviour of other submodules (e.g., enables registers, resets counters, switching multiplexers, etc.). This part of the design is called the *control path*. Write a state-output table for the FSM controller.

Hints:

- Keep the FSM simple. A set of 3 states IDLE, CAPTURE, and DONE could be sufficient, however, different approaches (e.g. with more states instead of just CAPTURE) are possible.
- The trickier part of the design is the tracking of the 8 heartbeat steps. Make sure you give this due consideration.
- Check your work against the names of input and output signals in the provided modules to ensure you have handled all relevant signals.

Step 3: add the FSM, review and complete your block diagram

Add the FSM controller and its interconnections to the block diagram. Visually review your block diagram for completeness (e.g., no unconnected signals) and clarity. Try to walk through the key operation steps to sense-check your work.

When ready, show your FSM design (state diagram and state-output table) and system RTL diagram to the tutor before proceeding to code.

Step 4: Create a Vivado project and assemble the design

Open Vivado and create a new project called `humanreceiver`. Import the module files given.

Modify the top module file `humanreceiver_top.v` (a stub is given), so that it:

- instantiates the necessary submodules and
- implements signal connections in accordance with your sketched RTL diagram.

A draft constraint file `humanreceiver_constraints.xdc` is given, check that the relevant I/O ports match your design and amend as necessary. Remember: All I/O ports in the top module must be constrained. Also, don' forget to check that you have the correct file set as *top* before running synthesis.

Step 5: Implement and test

Complete implementation and deploy the system to the board. Confirm correct operation of the system by entering a few viable and unviable 8-bit codes (refer to the decoder table to help you choose which symbols to attempt). Verify that captured bits match button presses, that LEDs blink correctly during capture, and that display persists correctly until next capture. Annotate your findings.

Well done! Please demonstrate the functionality of the system to the tutor to receive your grade.

 © ANU 2026

Part 2: A digital signal generator as a source [30 min, 5 marks]

This task is a short modification of Part 1. In this exercise, we do away with button press-entered bits and instead use a more realistic signal generator as a signal source. We get to use some lab testing equipment, which is always good engineering fun!

In this task, the "capture" and "signal" human inputs are no longer necessary, and neither is the 1Hz time base. Instead, an externally supplied signal `s_clock` provides the timekeeping for the other signal (`s_data`) to be captured by the SIPO register (i.e. the transmitter now provides the clock instead of the receiver as before). The serial clock line will only be active while valid data signals are provided, and be idle (fixed logic HIGH) otherwise. The "bursts" of 8 `s_clock` cycles will be at an arbitrary frequency, but a few orders of magnitude slower than the board's 100/125MHz, e.g., in the kHz range. See Figure 3.

The signals are designed so that `s_data` *updates with the negative edge of* `s_clock`*, while the read into the SIPO register should occur at the positive edge*. This is to ensure that the signal is suitably stable when being read.

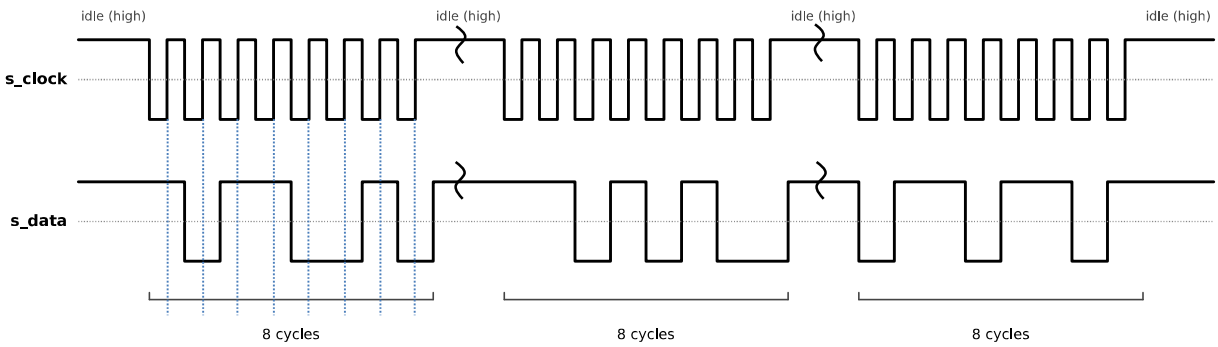

Figure 3: timing diagram exemplifying the nature of the external signals used in this task.
The dashed vertical lines mark the correct time instants for SIPO read (positive s_clock edge).

Here are some task steps:

Step 1: plan the necessary changes to the design

In your notebook, sketch changes to the RTL architecture that you think will be necessary for the system to work in the new configuration.

General guidance:

- Synchronise both `s_clock` and `s_data` to the system clock to avoid metastability.
- Connect the synchronised `s_clock` to the SIPO push/enable signal. Consider if SPOT may be needed here.
- Connect `s_data` to the serial input of the SIPO.
- Update the FSM controller if necessary to respond to `s_clock` instead of the 1Hz heartbeat.

The finer details will depend on how you have designed Part 1.

When ready, show your new intended design to the tutor before you proceed to coding it in Verilog

 © ANU 2026

Step 2: copy and modify the Part 1 design

Save a copy of the `humanreceiver` project and name it `autoreceiver`. Working with the new copy, make the following modifications to your Verilog code:

- Set up two PMOD pins as inputs to receive external signals `s_clock` and `s_data`. These need to be included both in the top module port list and in the constraint file.
- Remove the button logic and the 1Hz heartbeat logic
- Edit the top module so that the external signal lines go through synchronisers, and then onto the rest of your logic as per your sketch above.

Step 3: simulate your design

We have provided you with a generic testbench `TB_autoreceiver.v` which simulates one sequence of incoming signal at 10kHz. Use it to test your design and confirm correct operation.

Simulation tips:

- To avoid errors, confirm that the instantiation of `autoreceiver` in the testbench is consistent with your own implementation (e.g., I/O port names).
- Run the simulation for long enough to see the complete burst (~1ms)
- If you see any oddities in the top level response, remember that you can drag internal signals from your submodules to the simulation netlist an re-launch the simulation to probe internal behaviour and better appreciate what might be going on.
- Please ask for assistance from the tutor if you unsure of how to proceed. It is important to correct issues here before proceeding to implementation.

Please demonstrate to the tutor correct operation of the system when simulated.
This will be sufficient for you to receive the grades for the task.
You are encouraged to continue to the final hardware step, time permitting.

Step 4 (optional, time permitting): Implement and test your design

Run the Vivado implementation flow and ensure no unexpected errors or warnings show up.

Once ready, connect jumper wires to the PMOD ports you have chosen to use for the task. Connect a jumper wire to a ground pins as well. Refer to Figure 4 below for a refresher.

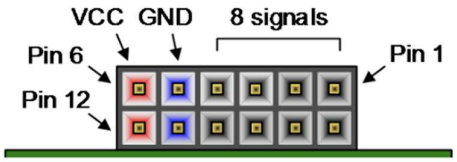

Figure 4: how PMOD pins are numbered on the Zybo Z7.

As always, please take care with jumper wires, so as to not accidentally short circuit the board: no jumper wires should be plugged in to VCC ports, and no wires should bridge directly the pinX ports into GND.

Connect the two signal generator channels to the system, so that channel 1 (`s_data`) and

 © ANU 2026

channel 2 (`s_clock`) are associated with the correct pins. You can also connect the oscilloscope to the same two ports if you want to observe the data and clock bursts.

Please consult the **separate reference document** provided, demonstrating the key connections and system operation steps for the signal generator used in this task.

Start the wave generation. If everything is working correctly, your system should display a meaningful message.

Please demonstrate to the tutor correct operation of the system when tested with the signal generator.

End of Lab 4

(please continue reading for Appendices)

Appendix: Trying this lab on a Basys3 board

The activities of this lab can be readily tested on a Basys3 board, as the necessary hardware (buttons, switches, LEDs) is very similarly set up, and available on both platforms. A few minor things to consider:

- **Seven-segment display differences:** cathodes and anodes are wired differently in the Basys3 vs the Zybo Z7 pmod module. This was discussed in detail in lab 3 and you should be able to reuse this knowledge to translate between one platform and the other with ease.
- **Hardware constraints.** As usual, these vary between boards. A reference constraints file for the Basys3 is available and a copy has been uploaded to Canvas.
- **Hardware opportunities.** There are more buttons, switches and LEDs on the Basys3. If you want, you can try and extend the task to include more controls and/or take advantage of the visual feedback from the LEDs, for example, visualising the received 8-bit ASCII code on some spare LEDs.

The final hardware testing will need to be done in-lab due to the need to use technical equipment. However, it is possible for you to practise the full design up to the simulation step of Activity 2 Part 2.

Please note that regardless of any preparation, all grading is done in-lab on Zybo boards.

 © ANU 2026